

Rangkuman Ponytail

Ponytail untuk Claude, Gemini, Antigravity, dan OpenCode + DeepSeek

1. Apa itu Ponytail?

Ponytail bukan model AI. Ponytail adalah layer berupa rules, skills, commands, dan adapter untuk AI coding agent. Tujuannya membuat agent lebih disiplin dalam menghindari over-engineering dan memilih implementasi yang sederhana serta memanfaatkan kode/infrastruktur yang sudah ada.

Prinsip utamanya: jangan membuat sesuatu yang tidak diperlukan; cek codebase terlebih dahulu; reuse existing code; gunakan standard library/platform/framework/dependency yang sudah tersedia; baru membuat implementasi baru jika memang diperlukan.

2. Apakah Ponytail hanya untuk Claude?

Tidak. Ponytail dirancang agar dapat digunakan oleh beberapa AI coding agent. Core behavior-nya tetap sama, sedangkan cara instalasinya mengikuti adapter masing-masing agent.

Agent	Cara instalasi	Catatan
Claude Code	<code>/plugin marketplace add DietrichGebert/ponytail</code> <code>/plugin install ponytail@ponytail</code>	Plugin Claude
Gemini CLI	<code>gemini extensions install https://github.com/DietrichGebert/ponytail</code>	Get ponytail extension
Antigravity	<code>agy plugin install https://github.com/DietrichGebert/ponytail</code>	Adapter Antigravity/Gemini
OpenCode	Tambahkan "@dietrichgebert/ponytail" pada <code>opencode.json</code>	Plugin OpenCode

3. OpenCode + DeepSeek V4 Flash

Ponytail dapat digunakan pada OpenCode tanpa bergantung pada Claude. Jika OpenCode dikonfigurasi menggunakan DeepSeek V4 Flash, Ponytail tetap berfungsi sebagai instruction/policy layer di atas agent.

Konfigurasi yang direkomendasikan:

```
{
  "$schema": "https://opencode.ai/config.json",
  "plugin": ["@dietrichgebert/ponytail"]
}
```

Model dapat diganti tanpa perlu menginstal ulang Ponytail, selama tetap menggunakan OpenCode environment yang sama.

4. Fungsi utama Ponytail

- Mengurangi over-engineering — mendorong agent memilih solusi yang proporsional.
- Reuse existing code — memeriksa service, component, utility, model, API, dan pattern yang sudah ada.
- Menghindari dependency baru — dependency hanya ditambahkan bila benar-benar diperlukan.
- Memanfaatkan native capability — menggunakan standard library, framework, dan platform sebelum membuat abstraction baru.
- Implementasi minimum — menyelesaikan requirement tanpa menambah kompleksitas yang tidak dibutuhkan.

- Review dan audit — menyediakan command untuk mengevaluasi diff dan repository.
- Menjaga engineering trade-off — pengurangan kode bukan tujuan jika mengorbankan security, validation, data-loss handling, atau accessibility.

5. Mode Ponytail

Mode	Kegunaan
lite	Rules lebih ringan; cocok untuk pekerjaan sederhana.
full	Default dan paling cocok untuk development sehari-hari.
ultra	Paling agresif menghindari kompleksitas dan cocok untuk audit/refactoring.
off	Menonaktifkan behavior Ponytail.

Contoh: `/ponytail full` untuk penggunaan harian atau `/ponytail ultra` saat ingin melakukan refactoring/audit yang sangat ketat.

6. Command yang penting

Command	Fungsi
/ponytail lite	Mengaktifkan mode ringan.
/ponytail full	Mode default untuk development.
/ponytail ultra	Mode paling ketat terhadap over-engineering.
/ponytail off	Menonaktifkan Ponytail.
/ponytail-review	Review current diff dan mencari implementasi yang bisa disederhanakan.
/ponytail-audit	Audit repository secara lebih luas.
/ponytail-debt	Mengelola/menelusuri technical debt yang ditandai.

7. Workflow yang direkomendasikan

Untuk workflow coding menggunakan OpenCode + DeepSeek, gunakan pola berikut:

1. Aktifkan full: /ponytail full
2. Berikan requirement: jelaskan fitur dan acceptance criteria.
3. Minta agent inspect terlebih dahulu: pahami architecture dan pattern yang sudah ada sebelum coding.
4. Implementasi: reuse existing code dan hindari dependency/abstraction baru tanpa alasan.
5. Review: jalankan /ponytail-review setelah perubahan selesai.
6. Refactor: sederhanakan bagian yang tidak diperlukan.

8. Contoh prompt untuk OpenCode + DeepSeek

Implement Budget Planner dengan requirement berikut:

- User dapat membuat budget.
- Budget memiliki category.
- User dapat mencatat expense.
- Tampilkan total budget, total expense, dan remaining budget.

Sebelum coding:

1. Inspect existing architecture dan pattern.
2. Reuse existing models, services, components, API pattern, dan utilities jika tersedia.
3. Jangan menambahkan dependency baru kecuali benar-benar diperlukan.
4. Implementasikan solusi minimum yang memenuhi requirement.

9. Rekomendasi setup

Untuk workflow multi-model, Ponytail sebaiknya ditempatkan pada level agent, bukan dianggap sebagai fitur khusus model. Contohnya:

```
OpenCode → Ponytail → DeepSeek V4 Flash
OpenCode → Ponytail → Claude Sonnet
OpenCode → Ponytail → Claude Opus
```

Dengan pola tersebut, Anda dapat mengganti model sesuai kebutuhan tanpa mengubah konsep Ponytail. DeepSeek dapat dipakai untuk pekerjaan coding rutin, Sonnet untuk pekerjaan yang lebih kompleks, dan Opus untuk reasoning/architecture yang membutuhkan kemampuan lebih tinggi.

10. Kesimpulan

Ponytail = engineering discipline layer untuk AI coding agent. Ia bukan model AI, bukan framework, dan bukan pengganti kemampuan model. Nilai utamanya adalah membuat agent lebih disiplin dalam membaca codebase, reuse existing solution, menghindari abstraction/dependency yang tidak perlu, dan menjaga implementasi tetap sederhana serta proporsional.

Untuk setup Anda, kombinasi OpenCode + Ponytail + DeepSeek V4 Flash merupakan penggunaan yang valid. Gunakan mode full sebagai default dan ultra ketika melakukan audit atau refactoring.

Referensi: repository Ponytail resmi — [DietrichGebert/ponytail](#)